Assignment 1

Reinforcement Learning Programming - CSCN 8020

Work mode: Individual assignment

Total value: 100 points

Submission mode: Submit only the required one-page PDF file to Brightspace. The full working solution must be available in a public GitHub repository.

Repository name: CSCN8020_Assignment1

Original Assignment Problems

Problem 1 [10]

Pick-and-Place Robot

Consider using reinforcement learning to control the motion of a robot arm in a repetitive pick-and-place task. If we want to learn movements that are fast and smooth, the learning agent will have to control the motors directly and obtain feedback about the current positions and velocities of the mechanical linkages.

Design the reinforcement learning problem as an MDP. Define the states, actions, and rewards, with reasoning.

Problem 2 [20]

Problem Statement

2x2 Gridworld

Consider a 2x2 gridworld with the following characteristics:

- State Space S: S = {s1, s2, s3, s4}
- Action Space A: A = {up, down, left, right}
- Initial Policy π: For all states, π(up | s) = 1
- Transition Probabilities P(s′ | s, a):
    - If the action is valid, meaning it does not run into a wall, the transition is deterministic.
    - Otherwise: s′ = s
- Rewards R(s):
    - R(s1) = 5 for all actions a.
    - R(s2) = 10 for all actions a.
    - R(s3) = 1 for all actions a.
    - R(s4) = 2 for all actions a.

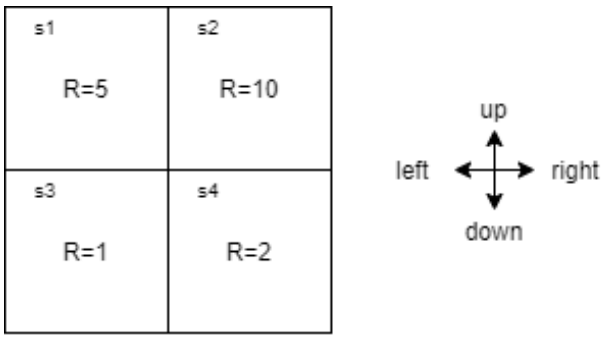

Figure 1: 2x2 Gridworld

Tasks

Perform two iterations of Value Iteration for this gridworld environment. Show the step-by-step process, without code, including policy evaluation and policy improvement. Provide the following for each iteration:

Iteration 1

1. Show the initial value function V for each state.
2. Perform value function updates.
3. Show the updated value function.

Iteration 2

Show the value function V after the second iteration.

Problem 3 [35]

Problem Statement

5x5 Gridworld

In Lecture 3's programming exercise, we explored an MDP based on a 5x5 gridworld and implemented Value Iteration to estimate the optimal state-value function V* and optimal policy π*.

The environment can be described as follows:

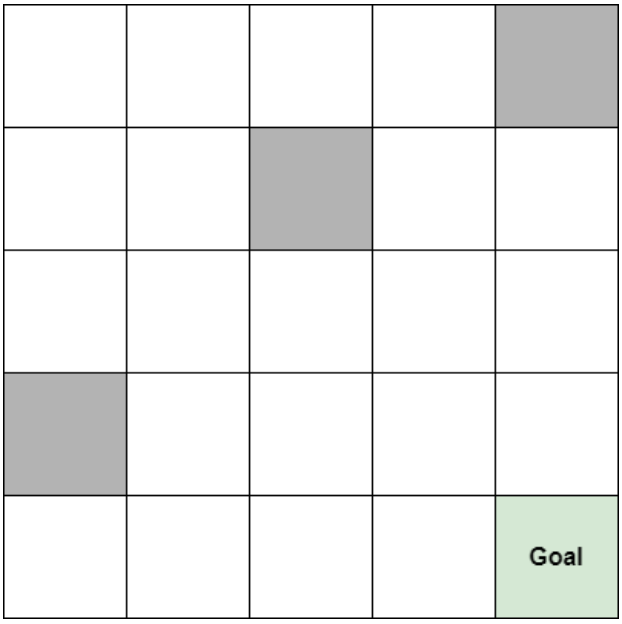

Figure 2: 5x5 Gridworld

- States: States are identified by their row and column, the same as a regular matrix. Example: the state in row 0 and column 3 is s0,3.
    - Terminal/Goal state: The episode ends if the agent reaches this state: sGoal = s4,4
    - Grey states: Sgrey = {s2,2, s3,0, s0,4}. These are valid but non-favourable states, as will be seen in the reward function.
- Actions: a1 = right, a2 = down, a3 = down, a4 = up for all states.
- Transitions: If an action is valid, the transition is deterministic. Otherwise: $s' = s$
- Rewards R(s):

Condition	Reward
If s = s4,4	+10
If $s \in$ Sgrey = {s2,2, s3,0, s0,4}	-5
If $s \in S$, $s \neq$ s4,4, and $s \notin$ Sgrey	-1

Task 1: Update MDP Code

4. Update the reward function to be a list of rewards based on whether the state is terminal, grey, or a regular state.
5. Run the existing code developed in class and obtain the optimal state-values and optimal policy. Provide figures of the gridworld with the obtained V* and π*. You may manually create a table.

Task 2: Value Iteration Variations

Implement the following variation of value iteration. Confirm that it reaches the same optimal state-value function and policy.

6. In-Place Value Iteration: Use a single array to store the state values. This means that you update the value of a state and immediately use that updated value in the subsequent updates.

Deliverables

- Full code with comments to explain key steps and calculations.
- Provide the estimated value function for each state.
- Important: Compare the performance of these variations in terms of optimization time, number of episodes, and provide comments on their computational complexity.

Problem 4 [35]

Problem Statement

Off-policy Monte Carlo with Importance Sampling

We will use the same environment, states, actions, and rewards in Problem 3.

Task

Implement the off-policy Monte Carlo with Importance Sampling algorithm to estimate the value function for the given gridworld. Use a fixed behavior policy b(a | s), such as a random policy, to generate episodes and a greedy target policy.

Suggested Steps

7. Generate multiple episodes using the behavior policy b(a | s).
8. For each episode, calculate the returns, meaning the sum of discounted rewards, for each state.
9. Use importance sampling to estimate the value function and update the target policy π(a | s).
10. You can assume a specific discount factor, such as γ = 0.9, for this problem.
11. Use the same main algorithm implemented in Lecture 4 in class.

Deliverables

- Full code with comments to explain key steps and calculations.
- Provide the estimated value function for each state.
- Important: Compare the estimated value function obtained from Monte Carlo with the one obtained from Value Iteration in terms of optimization time, number of episodes, computational complexity, and any other aspects you notice.

Additional Assignment Requirements

The original four problems above must be completed exactly as assigned. In addition, the following requirements apply to the full assignment submission.

1. Single Jupyter Notebook Requirement

Students must implement the solutions to all four problems in one single Jupyter Notebook.

The notebook must include Markdown cells that present the theory used in each problem, the mathematical explanations, the reasoning behind the Reinforcement Learning design choices, and the required talking points.

The notebook must also include code cells that show the actual Python implementation of the four problems, the algorithm execution, and the output tables, figures, logs, and results.

The notebook must be clear enough for the evaluator to run from top to bottom.

Important note for Problem 2: The original problem asks for the step-by-step process without code. Students must still show the manual mathematical process in Markdown. In addition, they must include a code implementation or code verification in the notebook as required by this updated assignment.

2. Required Development Library and Course Code References

Students are expected to use the Gymnasium development library as presented in the following course repository:

https://github.com/ProfEspinosaAIML/HelloGymMaze.git

Students are also expected to use the code samples provided with the course shell:

https://github.com/CSCN8020/playground/tree/main/lec2_MDP

https://github.com/CSCN8020/playground/tree/main/lec3_DP

https://github.com/CSCN8020/playground/tree/main/lec4_MC

Students may adapt the course code, but they must clearly explain their own implementation choices in the notebook.

3. Required Talking Points for Each Problem

For each of the four problems, students must include three talking points.

Talking Point A: Key Reinforcement Learning Feature

Describe a key feature of the Reinforcement Learning method being used to solve the problem. Examples may include Markov Decision Process design, state representation, action representation, reward design, policy evaluation, policy improvement, value iteration, Monte Carlo prediction, importance sampling, and behavior policy versus target policy.

Talking Point B: Implementation and Testing Challenge

Discuss one challenge found during the implementation and testing of the algorithm. Examples may include designing the reward function, handling invalid actions, avoiding infinite episodes, matching code output with expected mathematical results, comparing synchronous and in-place value updates, debugging policy updates, ensuring reproducibility, managing convergence criteria, and logging algorithm progress.

Talking Point C: Why This Is Reinforcement Learning

Explain why the implemented solution is a Reinforcement Learning algorithm and not another Machine Learning technique.

Students must map the Python code to the corresponding pseudocode presented in: Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

The explanation must connect the implementation to Reinforcement Learning concepts such as agent, environment, state, action, reward, return, policy, value function, episode, Bellman update, and exploration and behavior policy where applicable.

4. Required Software Architecture

A. Object-Oriented Design

The assignment must not be implemented as only loosely coupled Python scripts. Students must use object-oriented programming where appropriate.

Examples of expected classes may include GridWorldEnvironment, MDPProblem, ValueIterationAgent, InPlaceValueIterationAgent, MonteCarloAgent, Policy, ExperimentRunner, and Logger.

The exact class names are up to the student, but the design must show clear separation of responsibilities.

B. Log File Feature

The code must generate a log file showing the execution of the algorithm.

The log file should include useful information such as algorithm start and end time, parameter values, iteration progress, episode progress where applicable, final value functions and policies, and errors or warnings if applicable.

The log file must be included in the GitHub repository unless it is too large. If the log file is excluded, the notebook must show how to regenerate it.

5. GitHub Repository Requirement

Students must create a public GitHub repository using the following naming convention:

```
https://github.com/<USRID>/CSCN8020_Assignment1
```

Where <USRID> is the student's GitHub account ID. The repository must be public.

The repository must include the single Jupyter Notebook containing all four problems, all embedded assets required by the notebook, README.md, requirements.txt, .gitignore, any source files required to run the notebook, and any small supporting files required to reproduce the results.

The README.md file must include assignment title, student name, student ID, short summary of the assignment, instructions to run the notebook, description of the repository structure, and any assumptions or known limitations.

The requirements.txt file must allow the evaluator to replicate the development environment.

6. .gitignore and Repository Size Requirement

Students must add the virtual environment and any other runtime-generated sources to .gitignore. The repository must be quick to clone.

The .gitignore file must exclude files and folders such as:

```
.venv/

venv/

env/

__pycache__/

.ipynb_checkpoints/

*.pyc

*.pyo

*.log

.DS_Store
```

Additional large files, temporary files, cache folders, local datasets, and runtime-generated files must also be excluded when appropriate.

Important: The instructor will not wait for a long download. If the repository is slow to clone because virtual environments, large runtime files, cache folders, or other files that should have been listed in .gitignore were committed to the repository, the assignment will be graded with zero marks.

7. One-Page PDF Submission Requirement

Students must submit a one-page PDF file to Brightspace.

The PDF must include student name, student ID, assignment title, a 100-word summary of the work completed, and a link to the public GitHub repository.

The GitHub repository link must end in .git to allow easy automated cloning.

```
https://github.com/<USRID>/CSCN8020_Assignment1.git
```

Only the one-page PDF must be submitted to Brightspace. The complete assignment work must be available in the public GitHub repository.

8. Late Submission Policy

Late submissions will receive a 20% deduction from the final assignment score. This deduction is applied after the assignment is graded.

9. Academic Integrity and Use of External Resources

Students may use the course examples, textbook pseudocode, documentation, and Python libraries to support their work.

However, the submitted notebook must reflect the student's own implementation and understanding. Students must explain how their code works, cite or acknowledge any external references, libraries, tutorials, or code examples used, and must not submit unexplained code generated by another person, website, tool, or AI system.

Required Repository Structure

The repository should follow a clear structure similar to the following:

```
CSCN8020_Assignment1/
|
├── README.md
├── requirements.txt
├── .gitignore
├── CSCN8020_Assignment1.ipynb
|
├── src/
|   ├── environments.py
|   ├── agents.py
```

```
|   ├── policies.py
|   └── utils.py
|
├── images/
|   └── optional_diagrams_or_figures.png
|
└── logs/
    └── sample_execution.log
```

Students may use a different structure, but it must be clear, organized, and easy to evaluate.

Brightspace Submission Checklist

Before submitting, confirm that:

- [ ] The assignment was completed individually.
- [ ] The GitHub repository is public.
- [ ] The GitHub repository is named CSCN8020_Assignment1.
- [ ] The GitHub repository URL follows this format: https://github.com/<USRID>/CSCN8020_Assignment1.git.
- [ ] The repository clones quickly.
- [ ] The repository does not include a virtual environment.
- [ ] The repository includes a proper .gitignore.
- [ ] The repository includes one Jupyter Notebook with all four problems.
- [ ] The notebook includes Markdown explanations and code implementations.
- [ ] Each problem includes the three required talking points.
- [ ] The code uses object-oriented design.
- [ ] The code includes a logging feature.
- [ ] The repository includes README.md.
- [ ] The repository includes requirements.txt.
- [ ] The one-page PDF includes the required information.
- [ ] The one-page PDF is submitted to Brightspace.

Rubric

The assignment is worth 100 points.

Criterion	Excellent	Good	Satisfactory	Needs Improvement	Not Evident	Points
Problem 1: Pick-and-Place Robot MDP Design	Complete and well-reasoned MDP design with clear states, actions, rewards, and RL justification.	Mostly complete MDP design with minor gaps in reasoning.	Basic MDP design present but lacks detail or precision.	Incomplete or unclear MDP design.	Missing or not related to the problem.	10
Problem 2: 2x2 Gridworld Value Iteration	Correct two-iteration value iteration process with clear step-by-step explanation and accurate values.	Mostly correct with minor calculation or explanation issues.	Basic attempt with some correct updates but incomplete reasoning.	Significant errors in value updates or policy explanation.	Missing or not related to the problem.	20
Problem 3: 5x5 Gridworld Value Iteration and In-Place Variation	Correct implementation, value function, optimal policy, in-place variation, performance comparison, and complexity comments.	Mostly correct implementation with minor issues in comparison or explanation.	Basic implementation works partially but lacks depth in analysis.	Incomplete implementation or weak comparison.	Missing or not related to the problem.	35
Problem 4: Off-Policy Monte Carlo with Importance Sampling	Correct MC implementation with behavior policy, target policy, importance sampling, value estimates, and comparison with value iteration.	Mostly correct with minor issues in policy update, sampling, or comparison.	Basic MC implementation present but incomplete or weakly explained.	Major implementation or conceptual issues.	Missing or not related to the problem.	35
Single Jupyter Notebook Integration	One notebook integrates all four problems clearly with Markdown theory, math, explanations, code, outputs, and figures.	Notebook is mostly complete and organized, with minor clarity issues.	Notebook includes most required sections but is uneven or difficult to follow.	Notebook is disorganized or missing major explanatory elements.	Notebook missing or not runnable.	Included in problem scores
Required Talking Points	Each problem includes all three required talking points: RL feature, implementation/testing challenge, and why the solution is RL with reference to Sutton and Barto pseudocode.	Most talking points are complete, with minor gaps.	Talking points are present but superficial.	Talking points are incomplete or not clearly connected to the code.	Talking points missing.	Included in problem scores

Criterion	Excellent	Good	Satisfactory	Needs Improvement	Not Evident	Points
Object-Oriented Architecture	Clear object-oriented design with meaningful classes and separation of responsibilities.	Mostly object-oriented with minor design weaknesses.	Some classes used, but design is limited or inconsistent.	Minimal object orientation.	No object-oriented design.	Included in problem scores
Logging Feature	Meaningful log file or logging system shows algorithm execution, parameters, progress, and results.	Logging is present but limited.	Basic logging exists but lacks useful detail.	Logging is attempted but not useful or not reproducible.	No logging feature.	Included in problem scores
GitHub Repository, README, Requirements, and Assets	Public repository is complete, organized, easy to clone, and includes notebook, README, requirements, .gitignore, and all required assets.	Repository is mostly complete with minor omissions.	Repository is usable but missing some supporting detail.	Repository is difficult to use or missing important files.	Repository missing, private, or inaccessible.	Included in problem scores
One-Page PDF Submission	PDF includes name, student ID, assignment title, 100-word summary, and .git repository URL.	PDF is mostly complete with minor missing details.	PDF is submitted but incomplete.	PDF is unclear or missing major details.	PDF missing.	Included in problem scores

Automatic Penalties and Zero-Mark Conditions

Condition	Consequence
Late submission	20% deduction from the final assignment score
Repository is private or inaccessible	Assignment may be graded as incomplete
Repository URL does not end in .git	Submission may be delayed or marked incomplete
Missing one-page PDF in Brightspace	Assignment may be marked incomplete
Missing GitHub repository	Assignment may be marked incomplete
Missing Jupyter Notebook	Assignment may be marked incomplete
Missing README.md	Marks deducted according to severity
Missing requirements.txt	Marks deducted according to severity
Missing .gitignore	Marks deducted according to severity
Repository includes virtual environment, cache folders, large runtime files, or other files that should have been excluded, causing slow cloning	Zero marks
Notebook cannot be run because required files are missing	Marks deducted according to severity
Submission does not represent individual work	Academic integrity process may apply

---

Final Submission Instruction

Submit only the one-page PDF file to Brightspace.

The PDF must contain the public GitHub repository URL ending in .git.

The evaluator will clone the GitHub repository and use the notebook, README, requirements file, source code, assets, and logs to assess the assignment.